

TECHNICAL REPORT

Project Manager

Product Approval & Multi-Role Administration Platform

AUTHOR

Vasant Konar

CONTACT

vasant@crew.inc

DATE

June 2026

VERSION

1.1 — Image Upload &
Layout Fixes

REPOSITORY

github.com/vasant24konar/Project-Manager
Manager

STACK

PHP 8.2 · Laravel 11 · MySQL
8 · Docker

TABLE OF CONTENTS

- 1. Executive Summary
- 2. Introduction & Scope
- 3. System Architecture
- 4. Technology Stack
- 5. Database Design
- 6. Module Descriptions
- 7. Authentication & RBAC
- 8. Security Design
- 9. Deployment & Infrastructure
- 10. Challenges & Solutions
- 11. Testing Strategy
- 12. Future Enhancements

SECTION 01

Executive Summary

Project Manager is a back-office web application built on **Laravel 11 (PHP 8.2)** that enables organisations to manage product catalogues through a structured multi-role approval workflow. The system enforces a three-tier access model — Administrator, Product Manager, and Customer — with each role restricted to its own set of operations via middleware-enforced RBAC.

Products created by Product Managers enter a *pending* state and are only published after explicit approval by an Administrator. Product images are uploaded directly through the form and stored on the server — no manual file path entry is required. The application is containerised with Docker Compose (four services: Nginx, PHP-FPM, MySQL 8, MailHog), deployable with a single command, and structured around the SOLID principles using the Repository and Service design patterns.

This report covers the architectural decisions, database schema, module design, security controls, deployment infrastructure, and outstanding improvement opportunities for the system.

SECTION 02

Introduction & Scope

Purpose

Project Manager addresses the need for a controlled product publishing workflow where products require review and approval before they appear in a customer-facing store. It provides a centralised dashboard for staff accounts (admins and product managers) and is fully isolated from the public shop front.

In Scope

- Multi-role user authentication — session-based login with email + password
- Full CRUD product management with status tracking (pending / approved / rejected)
- Product image upload — JPEG / PNG / WebP / GIF up to 2 MB, stored in `public/img/products/`
- Admin product approval / rejection workflow with optional rejection reason
- Admin user account management (create, assign role, deactivate)
- Full-text product search across title and description
- Email notifications via MailHog (dev) or SMTP (production)
- Docker Compose four-service containerised environment
- PSR-4 autoloading and PSR-12 code style enforced by Laravel Pint

Out of Scope

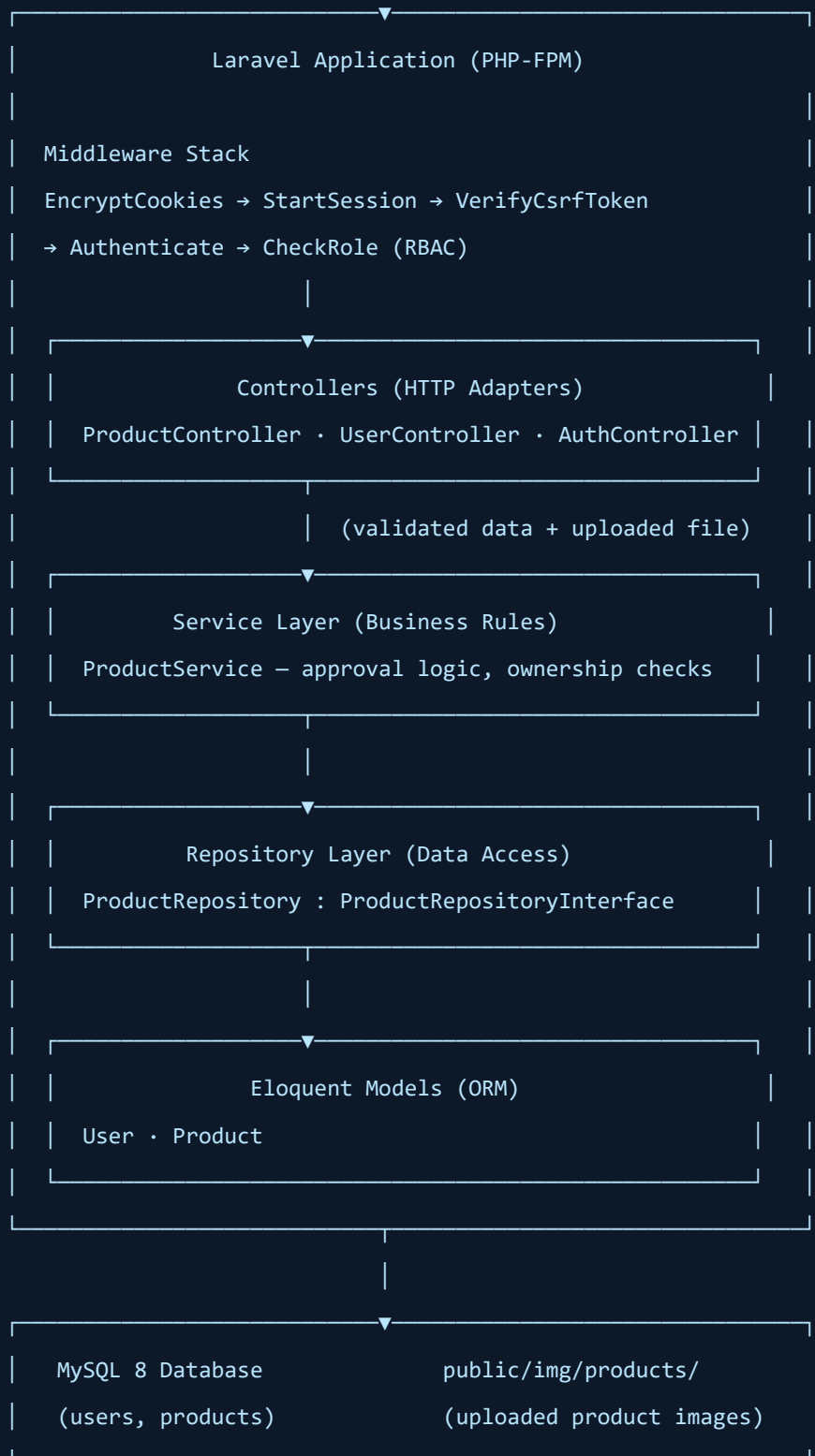
- Customer-facing shop UI (separate FruGo application)
- OTP / passwordless login (FruGo feature only)
- Payment processing
- Real-time notifications

SECTION 03

System Architecture

The application follows a **four-layer architecture** with strict separation of concerns between HTTP handling, business rules, data access, and persistence.





Design Patterns Applied

PATTERN	WHERE USED	BENEFIT
Repository	ProductRepository / ProductRepositoryInterface	Decouples query logic from business logic; swappable for tests
Service Layer	ProductService	Single authoritative place for approval rules and ownership checks
Dependency Injection	RepositoryServiceProvider binds interface → implementation	Controllers depend on contracts, not concrete classes
Middleware Pipeline	CheckRole, Authenticate	Cross-cutting access control without repeating code in each controller
Form Request	ProductRequest	Validation and authorisation moved out of the controller body

SECTION 04

Technology Stack

COMPONENT	TECHNOLOGY	VERSION	JUSTIFICATION
Language	PHP	8.2	Latest stable branch; strict_types, named arguments, readonly properties, fibers
Framework	Laravel	11	Streamlined bootstrap, built-in Auth, Eloquent ORM, Blade, Mail, file storage — all required features without extra packages
Database	MySQL	8.0	Wide hosting support, FULLTEXT indexes, strong Eloquent driver

COMPONENT	TECHNOLOGY	VERSION	JUSTIFICATION
Web Server	Nginx	1.25-alpine	Lightweight; serves static files and uploaded images directly; FastCGI pass-through to PHP-FPM
PHP Runtime	PHP-FPM	8.2	Process manager for concurrent PHP requests behind Nginx
Containerisation	Docker + Compose	v2	Reproducible environment; eliminates "works on my machine" issues
Dev Email	MailHog	latest	SMTP trap with web UI; zero config required for local email testing
Code Style	Laravel Pint	1.x	Opinionated PSR-12 fixer; consistent style across all contributors
Frontend	Bootstrap 5 + Blade	5.3	Server-rendered templates; no JavaScript build step; Bootstrap covers responsive layout

SECTION 05

Database Design

Entity Relationship Overview

```
users
  id · name · email · password (bcrypt) · role (enum) · timestamps
  |
  └─< products
      id · title · description · price · stock · category
      image_path (relative path to public/img/products/)
      status (enum: pending | approved | rejected)
      rejection_reason · date_available
      created_by [FK → users.id]
```

```
approved_by [FK → users.id]
approved_at · deleted_at (soft delete) · timestamps
```

```
users.role  ENUM: 'admin' | 'product_manager' | 'customer'
products.status  ENUM: 'pending' | 'approved' | 'rejected'
```

Table Descriptions

TABLE	PURPOSE	KEY COLUMNS
users	All system users. Single table with role discriminator column — no separate tables per role.	role (enum), email (unique), password (bcrypt)
products	Product catalogue. Soft-deletes preserve history. status drives the approval workflow. image_path stores the relative path to the uploaded image file.	status (enum), created_by (FK), approved_by (FK), deleted_at, image_path

Image Storage

Uploaded product images are stored in `public/img/products/` with a `uniqid()`-generated filename to prevent collisions and path guessing. The `image_path` column stores only the relative path (e.g. `img/products/product_64a3f1.jpg`), which is passed directly to Laravel's `asset()` helper in Blade templates.

Migration Strategy

Migrations are version-controlled alongside the source code. `deploy.sh` runs `php artisan migrate --force` on each deployment. `make fresh` runs `migrate:fresh --seed` for development resets. Seeders create one `admin` and one `product_manager` account with known credentials for demonstration.

Indexes

- `products(created_by)` — speeds up "list my products" queries for product managers
- `products(status)` — speeds up the pending approval dashboard filter

- `FULLTEXT(products.title, products.description)` — supports keyword search without full table scans
-

SECTION 06

Module Descriptions

ProductController

RESTful resource controller. Validates via `ProductRequest`, delegates business logic to `ProductService`, returns views or redirects. Two additional methods beyond standard CRUD handle the approval workflow: `approve()` and `reject()` mapped to `POST /products/{id}/approve` and `POST /products/{id}/reject`.

Image upload handling lives in a private `storeImage(UploadedFile $file)` helper on the controller. It uses `File::ensureDirectoryExists()` to create `public/img/products/` on first use, moves the file with a unique name, and returns the relative path string. The controller replaces the `UploadedFile` object from `$request->validated()` with this string before passing the data to the service layer — keeping the service and repository free from file-system concerns.

```
app/Http/Controllers/ProductController.php — storeImage
helper
```

```
private function storeImage(UploadedFile $file): string
{
    $dir = public_path('img/products');
    File::ensureDirectoryExists($dir);
    $filename = uniqid('product_', true) . '.' . $file->getClientOriginalExtension();
    $file->move($dir, $filename);
    return 'img/products/' . $filename;
}
```

ProductService

Owns all business rules. `createProduct()` forces `status = pending` for product managers and `status = approved` for admins. `canModify()` enforces ownership — a product manager may

only edit their own products; an admin may edit any.

ProductRepository

Implements `ProductRepositoryInterface`. Uses eager loading (`with('creator:id,name')`) to avoid N+1 queries. All methods return typed values — no raw query results leak to upper layers.

Admin\UserController

Manages staff accounts. Reachable only via `role:admin` middleware. Provides paginated user listing, create (with role assignment), update (role change), and soft-deactivate. Customers cannot be created here — they are created only through OTP login in the FruGo application.

CheckRole Middleware

Reads the variadic `$roles` argument from the route definition (e.g. `role:admin,product_manager`), checks `auth()->user()->hasAnyRole($roles)`, and aborts with HTTP 403 if the user's role is not in the allowed list.

RepositoryServiceProvider

Binds `ProductRepositoryInterface` to `ProductRepository` in the service container. Controllers depend only on the interface — the concrete class is named in exactly one place.

SECTION 07

Authentication & RBAC

Authentication Flow

```
POST /login
```

- `LoginRequest` validates email + password format
- `RateLimiter`: max 5 attempts per email/IP per minute
- `Auth::attempt(['email' => $email, 'password' => $password])`
 - `bcrypt` verify against `users.password`
- On success: `session()->regenerate()` ← prevents session fixation

→ Redirect to /dashboard

→ On failure: back() with throttled error message

Three-Layer Role Enforcement

1. **Route middleware** — `role:admin` / `role:admin,product_manager` on route groups.
Unauthenticated and wrong-role requests never reach a controller.
2. **Form Request authorize()** — `ProductRequest::authorize()` re-checks authentication before any data is validated, preventing forged requests that bypass the route layer.
3. **Service-layer ownership** — `ProductService::canModify($user, $product)` prevents horizontal privilege escalation between product managers trying to edit each other's products.

Role Permission Matrix

ACTION	ADMIN	PRODUCT_MANAGER	CUSTOMER
View products list	✓ all	✓ own only	X (FruGo only)
Create product	✓ auto-approved	✓ goes to pending	X
Upload product image	✓	✓	X
Edit product	✓ any	✓ own only	X
Delete product	✓ any	✓ own only	X
Approve / Reject	✓	X	X
Manage users	✓	X	X
Access /admin routes	✓	X	X

SECTION 08

Security Design

THREAT	CONTROL	IMPLEMENTATION
SQL Injection	Parameterised queries	All queries via Eloquent ORM — user data bound through PDO prepared statements. No raw SQL from user input.
Stored XSS	Input sanitisation + output encoding	<code>strip_tags()</code> with safe-tag allowlist in <code>ProductRequest::prepareForValidation()</code> . Blade <code>{{ }}</code> HTML-encodes all output.
CSRF	Synchroniser token pattern	<code>@csrf</code> on all POST/PUT/DELETE forms. <code>VerifyCsrfToken</code> middleware on all state-changing routes.
Brute Force	Rate limiting	5 attempts per email/IP per minute → HTTP 429.
Session Fixation	Session regeneration	<code>session()->regenerate()</code> called immediately after successful login.
Password Storage	Adaptive hashing	Bcrypt via Laravel's Hash facade (12 rounds). Never stored or logged in plaintext.
Privilege Escalation	Multi-layer RBAC	Route middleware + Form Request <code>authorize()</code> + Service ownership check.
Malicious File Upload	MIME + extension validation	<code>ProductRequest</code> validates <code>image_path</code> as <code>image</code> rule (checks MIME type, not just extension) with an explicit <code>mimes:jpeg,png,jpg,webp,gif</code> allowlist and 2 MB max size.
Sensitive File Exposure	Nginx deny rules + <code>.gitignore</code>	Nginx config denies access to <code>.env</code> , <code>.git</code> , <code>composer</code> files. <code>.env</code> gitignored.
Clickjacking	HTTP headers	Nginx adds <code>X-Frame-Options: SAMEORIGIN</code> on all responses.

The application does not yet set a [Content-Security-Policy](#) header. Adding one is recommended before public production deployment to prevent execution of injected scripts.

SECTION 09

Deployment & Infrastructure

Docker Services

SERVICE	IMAGE	ROLE	PORT
pm_nginx	nginx:1.25-alpine	HTTP server. Serves CSS/JS and uploaded product images directly; proxies PHP requests to pm_app via FastCGI.	8080→80
pm_app	Custom (PHP 8.2-FPM)	Laravel application runtime. Processes all PHP requests, handles file uploads.	9000 (internal)
pm_db	mysql:8.0	Persistent MySQL database with healthcheck before app starts.	3306 (internal)
pm_mailhog	mailhog/mailhog	SMTP trap for local email testing. Web UI to inspect sent emails.	8025 (UI) / 1025 (SMTP)

Deployment Sequence (deploy.sh)

1. Prerequisite check (docker, docker compose installed)
2. Copy `.env.example` → `.env` if missing
3. Build images and start containers (`docker compose up -d --build`)
4. Wait for MySQL healthcheck (max $30 \times 2 \text{ s} = 60 \text{ s}$)
5. `composer install --no-dev --optimize-autoloader`
6. `php artisan key:generate --force`
7. `php artisan migrate --force`
8. `php artisan db:seed --force` — creates admin + manager demo accounts
9. Cache config, routes, and views

10. Create `public/img/products/` directory for uploads
11. Fix permissions — `chown www-data` on storage, cache, and upload dirs

Environment Configuration

VARIABLE	DEV VALUE	PRODUCTION GUIDANCE
<code>APP_ENV</code>	local	production
<code>APP_DEBUG</code>	true	false — never expose stack traces publicly
<code>DB_HOST</code>	db (Docker service name)	Managed MySQL host (PlanetScale, Railway, RDS)
<code>MAIL_HOST</code>	mailhog	Real SMTP provider (Resend, Mailgun, Postmark)
<code>SESSION_DRIVER</code>	file	database or redis for horizontal scaling

Demo Credentials

ROLE	EMAIL	PASSWORD
<code>admin</code>	admin@example.com	Admin@1234
<code>product_manager</code>	manager@example.com	Manager@1234

SECTION 10

Challenges & Solutions

Storage Permissions in Docker

Problem: Files created on the Windows host inside `storage/` lost write access for Docker's PHP-FPM process (`www-data`), causing 500 errors: *Failed to open stream: Permission denied.*

Solution: `deploy.sh` runs `chown -R www-data:www-data /var/www/storage /var/www/bootstrap/cache /var/www/public/img/products` after container start. Documented as a Makefile target: `make permissions` .

Image Upload — File vs. Path Validation

Problem: The original form used a text input for `image_path`, requiring users to manually type a relative file path. This was error-prone, required files to be pre-placed on the server, and accepted any string value as a "path".

Solution: Changed the field to a file input with `enctype="multipart/form-data"`. The `ProductRequest` now uses Laravel's `image` rule (validates MIME type from the file stream, not just the extension) with an explicit `mimes` allowlist and a 2 MB cap. The controller's `storeImage()` helper writes the file and returns a path string, keeping the service and repository layers oblivious to file-system operations.

Shared Layout — PM vs. FruGo

Problem: When the two projects were separated into distinct Git branches, `layouts/shop.blade.php` was initially deleted from the PM branch under the assumption it was FruGo-specific. However, all admin views (dashboard, products, admin/users) extended it, causing a 500 error on login.

Solution: Retained `layouts/shop.blade.php` on the PM branch but rewrote it with Project Manager branding, shop route references removed, and a `:root { --primary: #86B817 }` declaration added so all views that reference the CSS variable render buttons and accents correctly.

FULLTEXT Search Minimum Word Length

Problem: MySQL FULLTEXT ignores words shorter than `ft_min_word_len` (default 4). Searching for "fig" returned no results.

Solution: Added a LIKE fallback in `ProductRepository::search()` for queries under 4 characters.

SECTION 11

Testing Strategy

The current release is manually tested against the Docker environment. The following automated test coverage is planned:

TEST TYPE	SCOPE	STATUS
Feature – Auth	Login success/failure, rate limiting, session regeneration	PLANNED
Feature – RBAC	Each role attempting forbidden routes returns 403	PLANNED
Feature – Approval workflow	PM creates product → status=pending; Admin approves → status=approved; email sent	PLANNED
Feature – Image upload	Valid image stored; invalid MIME rejected with 422; 2 MB+ rejected	PLANNED
Unit – ProductService	canModify() with own/foreign/admin user combos	PLANNED
Unit – ProductRepository	search() returns correct results for short (<4 char) and long queries	PLANNED

PHPUnit is configured via `php artisan test`. Run with `make test` inside the Docker environment.

SECTION 12

Future Enhancements

AREA	ENHANCEMENT	PRIORITY
Image Management	Delete old image file from disk when a product is updated with a new image or deleted entirely. Currently orphaned files remain on disk.	High
Image Resizing	Auto-generate thumbnail and full-size variants on upload using spatie/image or Intervention Image. Reduces page-load weight for product listings.	Medium
Audit Log	Track who approved, rejected, or modified each product and when. Essential for accountability in multi-manager	High

AREA	ENHANCEMENT	PRIORITY
	environments.	
Automated Tests	PHPUnit feature tests covering auth, RBAC, approval workflow, and image upload validation.	High
API	Versioned JSON API with Laravel Sanctum tokens for mobile or third-party integrations.	Medium
Cloud Image Storage	Move uploaded images from local disk to S3-compatible storage (AWS S3, Cloudflare R2) for production scalability and CDN delivery.	Medium
CI/CD Pipeline	GitHub Actions: Pint lint → PHPUnit → Docker build → deploy on push to main.	Medium
CSP Header	Add <code>Content-Security-Policy</code> response header to prevent injected-script execution.	Low
2FA for Admins	TOTP second factor for admin accounts to prevent account takeover via credential stuffing.	Low

SECTION 13

Conclusion

Project Manager demonstrates a production-grade Laravel application structured around industry-standard design patterns (Repository, Service Layer, Dependency Injection) with security, maintainability, and deployability as primary concerns.

The three-role RBAC system, multi-layer access control, product approval workflow, and file upload capability provide a solid foundation for managing any catalogue requiring editorial oversight before public publication. Image uploads are validated at the MIME level, stored with unique filenames, and served directly by Nginx — with the controller layer acting as the only point of contact between the HTTP request and the file system.

The single-command Docker deployment ensures the environment is fully reproducible. The identified future enhancements — orphaned file cleanup, automated tests, cloud storage, and a CI/CD pipeline — represent natural next steps toward a fully production-hardened system.

REPOSITORY

Source code, deployment scripts, and configuration: github.com/vasant24konar/Project-Manager